EEE404/591 Real Time DSP
Extra Credit Lab: Introduction to Neural Network Training

I. Objectives

The objective of this lab is to introduce students to the implementation of neural network on an embedded device such as on the STM32F407 board. After finishing this lab, the student will be familiar with:

- Neural network execution and training (forward and backward propagation).
- Training the neural network to implement a simple logic function system.

You will earn 10 points extra credit by completing this lab.

II. Hardware Setup

Set up the following hardware:

- STM32F407G-DISC1 kit: When you are ready to upload code to your board, connect USB Type-A end of the cable to your computer's USB port, connect the Mini-B end of the cable to the board. The PWR red LED will turn on.

III. Code

Download the following code from Canvas:

- main.c
- embeddedML.c
- embeddedML.h

IV. References

- STM32CubeIDE user guide (posted on Canvas)
- The EmbeddedML implementation used in this lab is downloaded from: https://github.com/merrick7/EmbeddedML

V. Tasks

Machine learning will be a critical technique for every engineering discipline. It offers the potential for remarkable advances in systems that can learn from and adapt to changing environments. As the number of standalone embedded devices expands at an exponential rate, many of them may be isolated or constrained by size or cost. It may be impossible or costly to pass the data collected from embedded devices to the "cloud" due to lack of available communication bandwidth and data center resources. Therefore, it becomes increasingly valuable to let embedded devices learn and act independently. One important step in this direction is to implement neural network execution and training on the embedded device itself without reliance on any other outside resource.

1. Neural Network Implementation Example

The neural network (NN) implementation used in this lab has 3 inputs, 1 hidden layer with 9 neurons, and 6 neurons in the output layer.

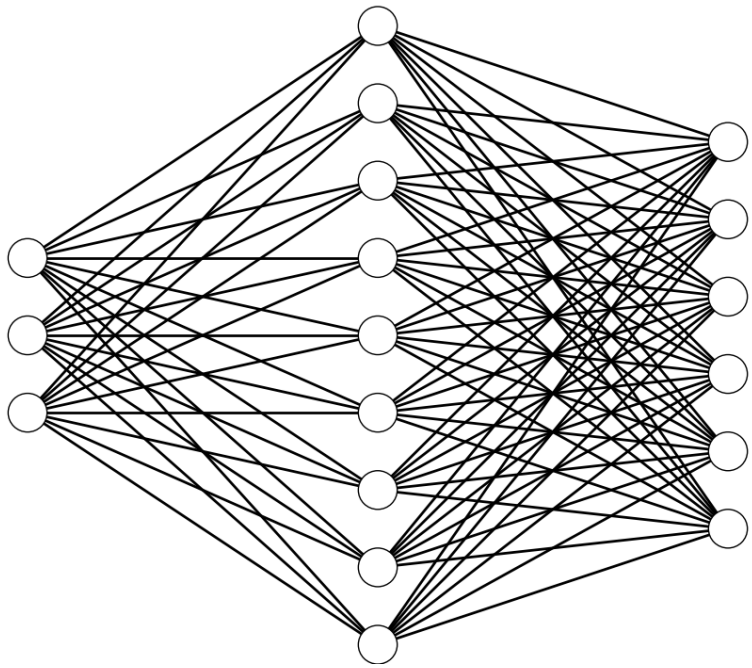


Figure 1: Neural Network Topology (3 inputs, 1 hidden layer with 9 neurons, 6 neurons in the output layer)

Recall the neural network individual neuron structure and operation discussed in lecture, for the NN topology shown in Figure 1, the training of the NN requires obtaining 3*9 + 9*6 = 81 weight values and 9 + 6 = 15 bias values.

As an example, the code is provided to train the NN to implement the following system:

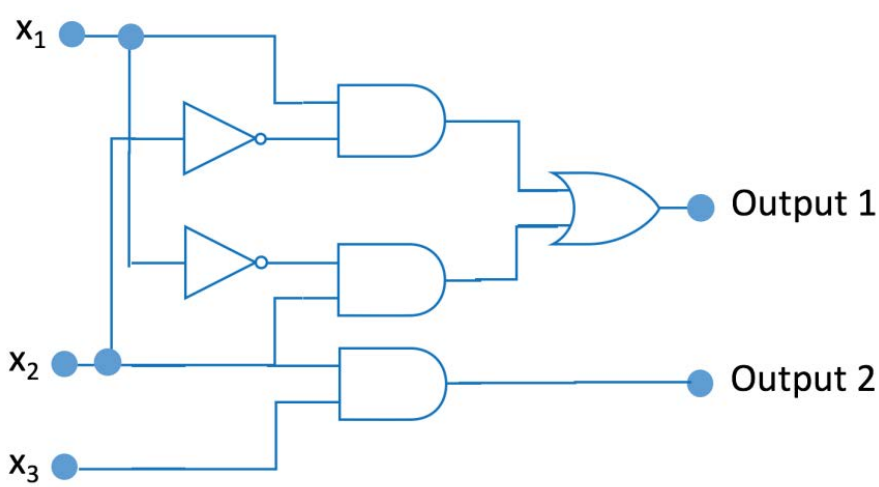


Figure 2: The 3-input, 2-output XOR-AND System

The truth table of the system in Figure 2 is shown below:

X_1	X_2	X_3	Output 1 (X_1 XOR X_2)	Output 2 (X_2 AND X_3)
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

Study the code to understand how the NN is trained and tested.

2. Neural Network Implementation Exercise

As an exercise, modify the example code to train the NN to implement the following system:

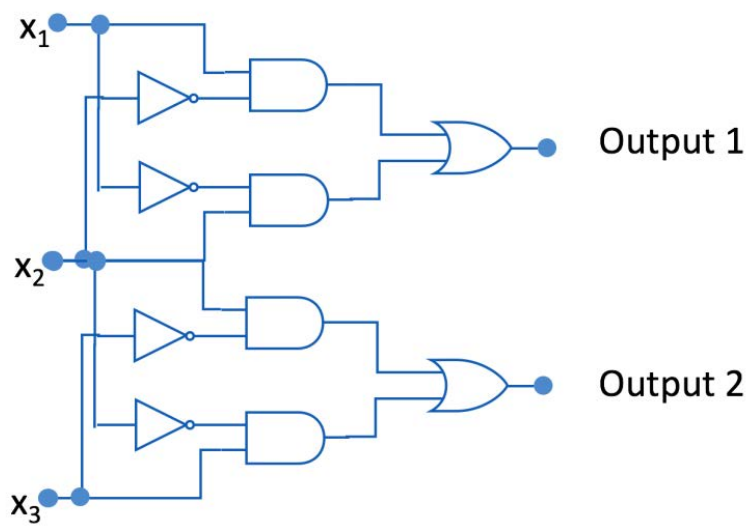


Figure 3: The 3-input, 2-output XOR-XOR System

The truth table of the system in Figure 3 is shown below:

X_1	X_2	X_3	Output 1 (X_1 XOR X_2)	Output 2 (X_2 XOR X_3)
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0

1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	0

Debug the code to make sure the NN training converges and the testing error is small. Prepare screenshot as instructed below.

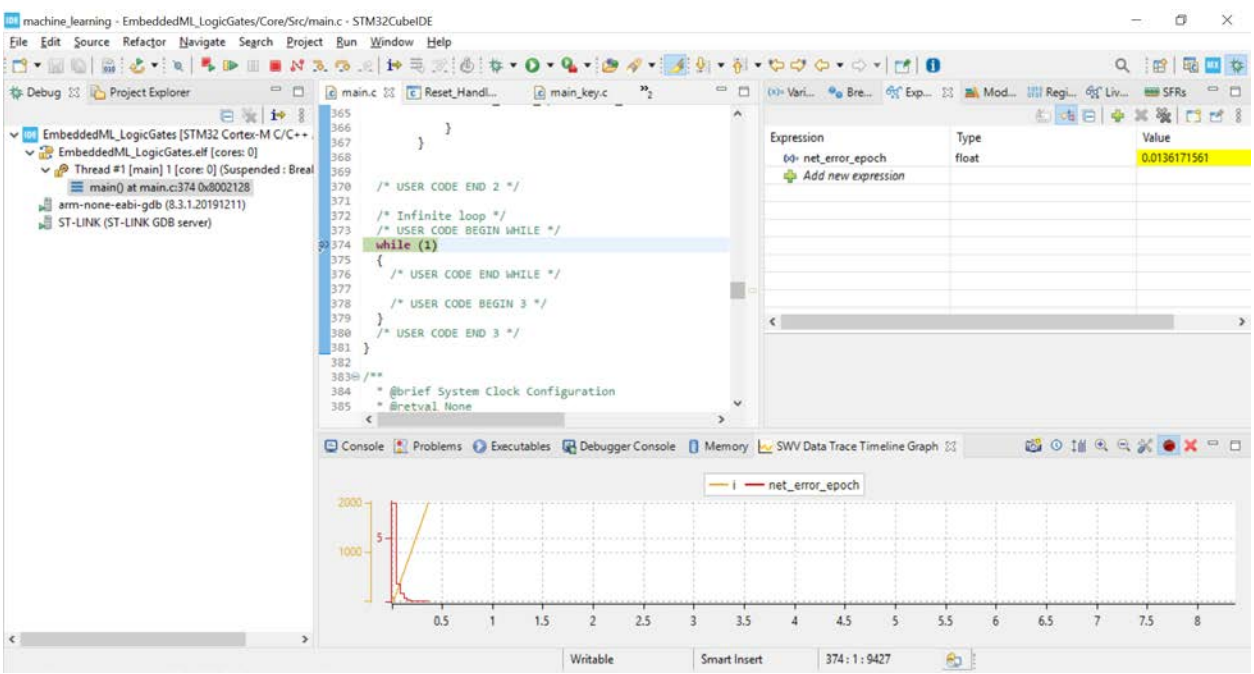
VI. Deliverables

Start your lab report by including the lab title and your name at the top.

Include in your lab report the followings. Submit your lab report as a single PDF file to Canvas.

1. Screenshots (1): screenshot of the IDE window (on Windows machine, use CTRL+ALT+PRT SC to take a screenshot of a window). You need to make sure the screenshots are clearly readable.

Program stops at a breakpoint in a function. Variable window shows the current value of net_error_epoch. Note your number should be different from the screenshot below. SWV data trace timeline graph shows the evolution of i (training epochs) and net_error_epoch.



2. Code: briefly explain what modifications you applied to train the new system. Include the part of the code that you modified.